API Reference

Base: {host}/api/ai. Every JSON response uses the envelope { success, data, error: { code, message } | null, traceId } — except POST /chat/stream (SSE frames). Error codes:

HTTP	Code	Meaning
400	VALIDATION_ERROR	Empty/oversize input, unknown language/provider
400	UNKNOWN_PROVIDER	Bad /selection payload
503	PROVIDER_NOT_CONFIGURED	Provider selected but not registered / key missing
502	AI_PROVIDER_UNAVAILABLE / TRANSLATION_PROVIDER_UNAVAILABLE	Upstream failure (safe message, no internals)
500	INTERNAL_ERROR	Unexpected (generic message only)

POST /chat — non-streaming chat

Request { message, history?[{role,content}], responseLanguage?, provider?, sessionId? }:

```
curl -X POST {host}/api/ai/chat -H "Content-Type: application/json" \
  -d '{"message":"what are your services?","sessionId":"demo-1"}'
```

Response data: { reply, provider, model, responseLanguage, sessionId }. With sessionId, history persists server-side (IConversationStore); without it, pass history per call (stateless). responseLanguage must be in the Languages:Supported allow-list.

POST /chat/stream — SSE streaming

Same body. Frames: data: <delta> … data: [DONE]. Errors mid-stream: data: {"error":"…"}. No envelope (SSE).

POST /translation/translate

{ text, targetLanguage, sourceLanguage?, provider? } → data: { translatedText, targetLanguage, detectedSourceLanguage, provider }.

POST /translation/batch — 1–100 texts

{ texts[], targetLanguage, sourceLanguage?, provider? } → data: { translatedTexts[], targetLanguage, provider }.

POST /translation/detect-language

{ text, provider? } → data: { language, confidence, provider }.

GET /translation/languages — provider's language list (envelope).

GET /providers — { chatProviders[], translationProviders[] } (safe to expose).

GET /info — effective providers, services[] (`name, provider,

apiKeyConfigKey, apiKeyConfigured`), languages. Booleans/pointers only — never endpoints, models, or secret values.

GET /selection — current effective selection + available providers.

POST /selection — switch live, no restart

`{ chatProvider?, translationProvider? }` (null/`"default"` = configured default). Unknown names → 400 `UNKNOWN_PROVIDER`. **Protect with admin auth in production.**

Settings — password-gated (`X-Settings-Token` header)

Mounted only when `AiSuite:Settings:Enabled`. 401 code `UNAUTHORIZED` on a missing/expired token (envelope). Full error payloads for upstream failures may arrive as ProblemDetails (`title` holds the message) instead of the envelope — handle both.

Route	Body / notes	
`POST /settings/login`	`{ password }` → `data: { token, expiresAtUtc }`	
`POST /settings/logout`	revokes the token	
`GET /settings`	snapshot: `security`, `chatProvider`, `llmKeys[]` (masked + `status.state`), `voice` (masked), `documents[]`, `vendorPresets[]`	
`PUT /settings/password`	`{ currentPassword, newPassword }`	
`POST /settings/llm-keys`	`{ vendor, endpoint?, model?, apiKey?, name?, maxTokens?, temperature?, enabled? }` → masked entry (omit endpoint/model = preset defaults)	
`PUT /settings/llm-keys/{id}`	partial patch + `move: "up"\`	"down"; **apiKey` null/omitted = keep stored**
`DELETE /settings/llm-keys/{id}`	`{ deleted: true }`	
`POST /settings/llm-keys/{id}/test`	live round-trip → `{ ok, latencyMs, statusCode, model, message }` (secret-free; writes health/cooldown)	
`PUT /settings/voice`	`{ sttEnabled, sttProvider, sttEndpoint?, sttModel?, sttApiKey?, ttsEnabled, ttsProvider, ttsEndpoint?, ttsModel?, ttsVoice?, ttsApiKey? }` — **blank key = keep stored**	
`POST /settings/kb`	multipart `file` (.pdf/.docx/.md/.txt, ≤20 MB) → `{ document, characters, warnings }`	
`DELETE /settings/kb/{id}`	`{ deleted: true }`	

Runtime voice — public when enabled in settings

Route	Notes
`POST /voice/stt`	multipart `file` + optional `language` → `data: { text, provider }`
`POST /voice/tts`	`{ text, voice?, language? }` → **raw audio bytes** (content-type from provider, not the envelope)

Disabled providers answer `VOICE_UNAVAILABLE` (400/503 envelope).